Availability Calculations

series multiplies, parallel adds nines

Watch the 5-min video

In one line: components in series multiply their availabilities, so every hard dependency lowers yours; redundant components in parallel fail only when all fail, so availability becomes $1 - (1 - A)^n$ — as long as the failures are really independent.

1 A promise you can't keep

Your service promises **99.99%**. It calls five services, each at **99.9%**. Even if your own code never fails, $0.999^5 \approx$ **99.5%**: about 44 hours of downtime a year against a promise of 53 minutes. You can't be more available than the things you depend on, unless you add redundancy or remove the dependency from the critical path.

2 What the nines mean

Availability	Downtime per year	Per month	Typical for
99% (two nines)	3.65 days	7.2 hours	internal tools, batch jobs
99.9% (three nines)	8.8 hours	43 minutes	most web products
99.99% (four nines)	53 minutes	4.3 minutes	payments, core APIs
99.999% (five nines)	5.3 minutes	26 seconds	telecom, critical infrastructure

Availability = uptime ÷ total time. Each extra nine cuts allowed downtime by 10×, and usually costs far more than 10× the engineering.

3 The two formulas

$$\text{series: } A = A_1 \times A_2 \times \dots \times A_n \quad \cdot \quad \text{parallel: } A = 1 - (1 - A)^n$$



Figure 1. Series dependencies multiply; parallel replicas multiply only the **failure** probabilities.

Series means a request needs every component, so you multiply. **Parallel** means one healthy copy is enough: the system is down only when all copies are down together, so you multiply the failure

probabilities (1 − A) and subtract from 1. Two 99% replicas give $1 - 0.01^2 = 99.99\%$. Redundancy adds nines; dependencies remove them.

WORKED EXAMPLE: THREE REPLICAS AT 99.9%

Each replica is down with probability 0.001. All three are down together with probability $0.001^3 = 10^{-9}$, so the replica tier is theoretically **99.9999999%** (nine nines). Put a 99.99% load balancer in front and the path is $0.9999 \times 0.999999999 \approx$ **99.99%**: the load balancer now sets the ceiling. And one bad deploy that hits all three replicas at once makes the real number far lower than nine nines.

4 Why real numbers are lower



Figure 2. The parallel formula assumes independent failures. These break that assumption.

- **Correlated failure:** replicas in one zone, built from one image, or changed by one deploy fail together.
- **Ceilings:** a hard dependency in series caps you at its availability, however many replicas you add elsewhere.
- **Recovery time:** downtime ≈ how often you fail × how long you take to recover (MTTR). Faster detection and rollback raise availability too.

HOW TO RAISE AVAILABILITY

- Remove dependencies from the critical path: make them asynchronous or optional.
- Add redundancy across **independent** failure domains: zones, then regions.
- Add fallbacks and caches so a failed dependency degrades the answer instead of breaking it.
- Cut time to detect and recover: health checks, automatic failover, fast rollback.

COMMON MISTAKES

- Promising four nines on top of three-nines dependencies.
- Counting replicas as independent when they share a zone, a deploy or a database.
- Forgetting the load balancer, DNS or gateway that sits in series with everything.
- Measuring uptime of servers instead of success of user requests (lesson 2).

5 Key terms

Availability: fraction of time (or requests) the system works

Nines: shorthand: 99.9% = three nines

Series: all components needed: multiply availabilities

Parallel: any one component enough: $1 - (1 - A)^n$

Failure domain: set of things that fail together

MTTR: mean time to recover

6 Interview questions

Q1. What does 99.9% availability allow? BASIC

About 8.8 hours of downtime a year, or roughly 43 minutes a month.

Q2. Your service depends on three services at 99.95% each. What is the best you can offer? BASIC

In series you multiply: $0.9995^3 \approx 99.85\%$, before counting your own failures.

Q3. Two replicas at 99% each behind a 99.99% load balancer. What's the availability? SCENARIO

Replicas: $1 - 0.01^2 = 99.99\%$. In series with the LB: $0.9999 \times 0.9999 \approx 99.98\%$.

Q4. How would you raise a service from 99.5% to 99.95%? SCENARIO

Find the weakest dependencies in the critical path; make some async or optional, add fallbacks, spread replicas across zones, and shorten detection and recovery.

Q5. Why don't three replicas give you nine nines in practice? TRICKY

Failures are correlated: shared zones, deploys, configs and dependencies take replicas down together, and the components in series cap the total.

Q6. Is availability better measured in time or in requests? TRICKY

Requests usually: a request-based SLI (good requests ÷ total) reflects what users feel, including partial outages that time-based uptime hides.

Next: SD-K-002 · SLIs, SLOs and SLAs backendroadmap.com